

웹 취약점 진단 보고서

Web Application Vulnerability Assessment Report

1. 진단 사이트 개요

진단 호스트	localhost / Target Server (Masked)
종합 진단 결과	취약 발견 (VULNERABLE)

본 보고서는 마스킹 처리된 패킷 데이터를 기반으로 LLM이 진단한 결과 및 N/A 판정 근거를 포함하며, 개발 언어 미특정 환경을 고려하여 주요 프레임워크별 시큐어코딩 가이드를 제공합니다.

2. 진단 결과 요약 및 상세 분석

취약점 명칭	SQL Injection
OWASP 분류	A05 Injection
위험도 (Severity)	HIGH
판단 근거 요약	요청의 'searchstring' 파라미터에 SQL 주입 공격 패턴이 포함되어 있으며, 응답에서 해당 입력이 그대로 반영되어 SQL 주입 가능성이 높습니다.
요청 분석 근거 (Request)	• 'searchstring' 파라미터에 SQL 주입 공격 패턴인 ") order by 1#"가 포함되어 있습니다.
응답 분석 근거 (Response)	• 응답 본문에서 '상품명 : ') order by 1#'로 검색하셨습니다.'라는 문구가 있어 입력이 그대로 반영되었음을 알 수 있습니다.
보안 영향 (Impact)	공격자가 데이터베이스의 구조를 파악하거나 민감한 정보를 탈취할 수 있습니다.
추가 확인사항 (N/A 전용)	None

3. 프레임워크별 시큐어코딩 및 패치 가이드

대응 방안 요약: 사용자 입력을 철저히 검증하고, SQL 쿼리 작성 시 매개변수를 사용하여 SQL 주입을 방지하십시오.

Spring (Java)

취약한 코드 (Vulnerable)

```
String query = "SELECT * FROM users WHERE user_id = " + userId + "";  
Statement stmt = connection.createStatement();
```

안전한 패치 코드 (Secure)

```
String query = "SELECT * FROM users WHERE user_id = ?";  
PreparedStatement pstmt = connection.prepareStatement(query);  
pstmt.setString(1, userId);
```

Flask (Python)

취약한 코드 (Vulnerable)

```
query = f"SELECT * FROM users WHERE user_id = '{user_id}'"  
cursor.execute(query)
```

안전한 패치 코드 (Secure)

```
query = "SELECT * FROM users WHERE user_id = %s"  
cursor.execute(query, (user_id,))
```

Node.js

취약한 코드 (Vulnerable)

```
const query = `SELECT * FROM users WHERE user_id = '${userId}'`;  
db.query(query);
```

안전한 패치 코드 (Secure)

```
const query = 'SELECT * FROM users WHERE user_id = ?';  
db.query(query, [userId], callback);
```